PROGRAMMING FUNDAMENTALS

CS – 101

Instructor: Maria N. Samad

October 12th, 2022

October 14th, 2022

UPDATING VARIABLES

- There are various ways in which a variable can be modified:
  - **Update**:
    - The new value of the variable depends on the older/previous value
    - For example:
      - `x = x + 1`
      - `y = y**2`
      - `z = z + x + 1`
  - **Initialize**:
    - Defining a variable and assigning some starting value in it
    - For example:
      - `x = 0` → Initialize
      - `x = x + 1` → Update
      - `y = 3` → Initialize
      - `y = y**2` → Update
      - `z = x + 1` → Neither
      - `z = x` → Initialize

UPDATING VARIABLES

- There are various ways in which a variable can be modified:
  - **Increment**:
    - Changing the value of a variable by adding 1 to it
    - For example:
      - `x = x + 1`
      - `y = y + 1`
      - `z = z + 1`
  - **Decrement**:
    - Changing the value of a variable by subtracting 1 from it
    - For example:
      - `x = x - 1`
      - `y = y - 1`
      - `z = z - 1`

ITERATION

- Commonly defined as ***loops***
- A process in which a set of instructions are repeated in a sequence for a specified number of times, or until a condition is met
- Each set of execution cycle is called ***iteration***

TYPES OF LOOPS

- Two main categories of loops:
  1. **<u>Entry Controlled Loops</u>**:
     - The test condition to execute the loop is tested **BEFORE** entering into the loop
     - Thus, loop will not even enter the body or executed even once if condition is not satisfied
     - In other words, it's a condition to **"enter"** the loop, hence the name
     - Examples:
       - ***For Loop***
       - ***While Loop***

TYPES OF LOOPS

2. **<u>Exit Controlled Loops</u>**:
   - The test condition to execute the loop is tested **AT THE END** of the loop
   - Hence, loop will be executed at least once, irrespective of the condition
   - In other words, it's a condition to **"exit"**, i.e. end the loop
   - Examples:
     - ***Do-while Loop***

- Python only implements ***for*** and ***while*** loop.
- All loops can be used to define:
  - Infinite loops
  - Nested loops

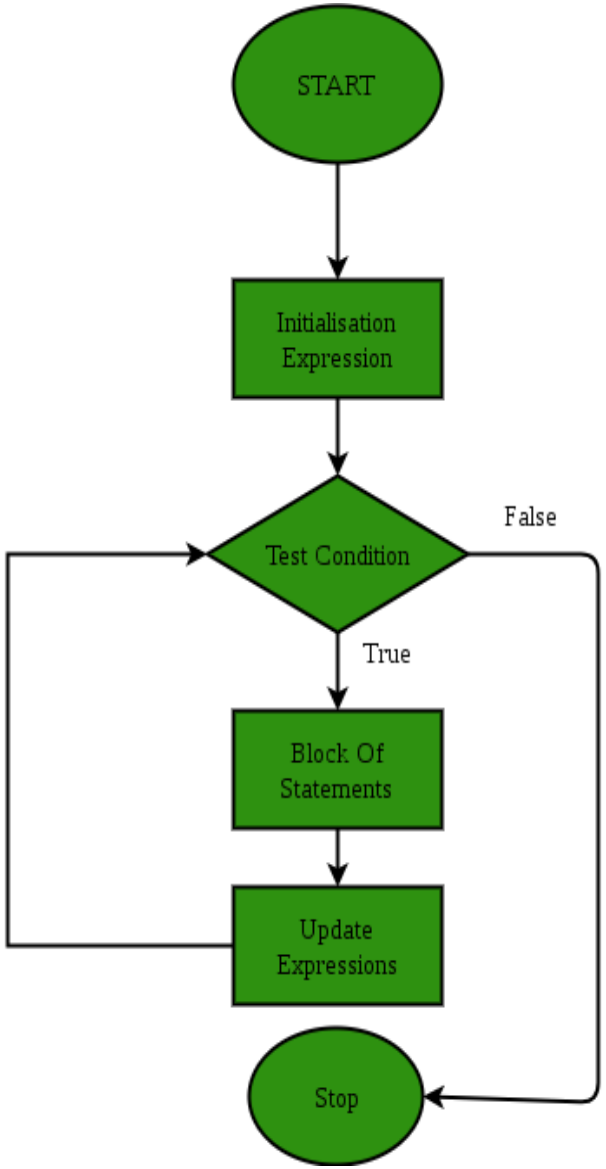
FOR LOOP

- Allows to write a piece of code that is executed a specific number of times

- **Syntax**:
  - for loop_variable in sequence:
    statement(s)
    statement(s)

- Sequence is iterated over a range of values or the actual values

FOR LOOP


START
Initialisation Expression
Test Condition
False
True
Block Of Statements
Update Expressions
Stop

FOR LOOP

- Three major elements of a loop:
  1. **Initialize expression**:
     - Defines and initializes loop counter variable(s)
  2. **Test Expression**:
     - Test a specified condition to enter the loop
  3. **Update Expression**:
     - After loop statements are executed, loop counter variable(s) must be updated

FOR LOOP

- For loops can be iterated over a range of values, as well as the actual values:
  - ***Range***:
    - Specifies the starting and ending indices of the loop counter variable
    - For example, how many times does the loop run?
  - ***Iterators***:
    - Uses a sequence of elements to specify the start and end of the loop
    - For example, strings and lists, etc

EXAMPLE (RANGE OF VALUES)

- n = 4
  ```
  for  index  in  range  (0, n):
          print(index)
  ```
- This loop will run over the range of values from 0 to n – 1 (both inclusive)
- Output?

EXAMPLE (RANGE OF VALUES)

- n = 4
  ```
  for  index  in  range  (n):
          print(index)
  ```
- This loop will run over the range of values from 0 to n – 1 (both inclusive)
- If starting point is always 0 then no need to specify it in the code
- Output?

EXAMPLE (RANGE OF VALUES)

- n = 4
  for index in range (0, n, +2):
  print(index)
- This loop will run over the range of values from 0 to n – 1 (both inclusive) with the change of values as +2 so 0, 2, 4, 6, and all the way to n – 1
- We can use it with any starting point as well, for example:
  - for index in range (1, n, +2)
- Output?

EXAMPLE (RANGE OF VALUES)

- Can directly use numbers as well, to specify the range
- for index in range (4):
  print(index)
- Output?

- for index in range (1, 4):
  print(index)
- Output?

EXAMPLE (RANGE OF VALUES)

- Can count down as well, to specify the range
- for index in range (4, 0, -1):
  print(index)
- Output?

EXAMPLE (ITERATORS)

- Iterators are sequence of elements stored/written together
- For example, lists and strings
  - **List**:
    - A sequence of elements having the same type
    - Also called ***array***
    - For example:
      - arr1 = [1, 2, 3, 4]
      - arr2 = [“Hello”, “There”, “How”, “Are”, “You”]
      - arr3 = [‘A’, ‘G’, ‘U’, ‘F’, ‘T’]
      - arr4 = [True, True, False, True, False]
      - arr5 = [2.5, 3.13, 4.67]

EXAMPLE (ITERATORS)

- **String**:
  - An indirect form of list containing all characters
  - No need to comma-separate them like in lists
  - For example:
    - str1 = “Hello”
    - str2 = “Hello World”
    - str3 = “Pi is 3.14”
    - str4 = “True” → NOT BOOLEAN

EXAMPLE (ITERATORS - LIST)

- arr1 = [1, 2, 3, 4]

```
for index in arr1:
        print(index)
```

- This does not iterate over the positions of the elements but the actual values of the list elements
- Output?

EXAMPLE (ITERATORS - LIST)

- arr2 = [“Hello”, “There”, “I’m”, “HAPPY”]

```
for index in arr2:
        print(index)
```

- Output?

EXAMPLE (ITERATORS - STRING)

- str1 = “Programming Fundamentals”

```
for index in str1:
        print(index)
```

- Output?

EXAMPLE (RANGE & ITERATORS)

- list1 = [“Apple”, “Banana”, “Cherry”, “Grape”, “Strawberry”, “Blueberry”, “Orange”]

```
for index in range(len(list1)):
        print(list1[index])

for index in list1:
        print(index)
```

- Output?

EXERCISES

1. Write a function that calculates Factorial value of n using loops, where n > 2
2. Write a function that calculates the power of some base ‘b’ w.r.t to some exponent ‘e’ value using loops
3. Write a function to print the first ‘n’ numbers of Fibonacci series, separated by a comma, where n > 2, i.e.

   0, 1, 1, 2, 3, 5, 8, 13, 21, …

   (The last number is printed without the comma)
4. Write a function that prints the index and the element of the list, ‘lst1’ in reverse order using single loop

EXERCISE # 1 SOLUTION

1. Write a function that calculates Factorial value of n using loops, where n > 2

- def fact(num):
    factorial = 1
    if num == 0:
        factorial = 1
    else:
        for i in range(1, num + 1):
            factorial = factorial * i
    return factorial

- n = 1
  print(fact(n))

EXERCISE # 2 SOLUTION

2. Write a function that calculates the power of some base ‘b’ w.r.t to some exponent ‘e’ value using loops, where n >= 0

- def power(b,e):

```
        result=1
        for i in range(e, 0, -1):
                result *= base
        return result
```

- base = 3

  exp = 3

  print(power(base, exp))

EXERCISE # 3 SOLUTION

3. Write a function to print the first ‘n’ numbers of Fibonacci series, separated by a comma, where n > 2

- def fib (n):

```
    num1 = 0
    num2 = 1
    for i in range(n - 1):
        print(num1, end=", ")
        res = num1 + num2
        num1 = num2
        num2 = res
    print(num1)
```

- fib(8)

EXERCISE # 4 SOLUTION

4. Write a function that prints the index and the element of the list, ‘lst1’ in reverse order

- def rev_ord(lst1):
        for i in range(len(lst1)-1, -1, -1):
            print(i, "-", lst1[i])

- l1 = [30,40,50,60,70]
  rev_ord(l1)